Sesion 13

Descomposición en Valores Singulares (SVD)

Carlos Cesar Sanchez Coronel

CALCULO Y ALGEBRA LINEAL PARA IA

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Enunciado del teorema

SVD (Singular Value Decomposition)

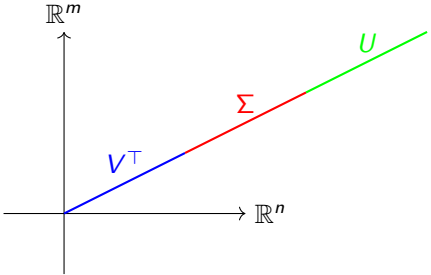
Toda matriz $A \in \mathbb{R}^{m \times n}$ de rango r se puede factorizar como

$$A = U \Sigma V^{\top}$$

donde:

- $U \in \mathbb{R}^{m \times m}$ ortogonal: columnas = vectores singulares izquierdos.
- $V \in \mathbb{R}^{n \times n}$ ortogonal: columnas = vectores singulares derechos.
- $\Sigma \in \mathbb{R}^{m \times n}$ diagonal: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ (valores singulares).

Interpretacion geometrica



- $V^\top$: rotacion en el espacio de entrada.
- Σ: escalado por σ_i (y cambio de dimension).
- U: rotacion en el espacio de salida.

Forma reducida (economy SVD)

- Cuando $m > n$, la SVD completa tiene muchos ceros en Σ.
- SVD reducida: $A = U_r \Sigma_r V^\top$, con $U_r \in \mathbb{R}^{m \times r}$, $\Sigma_r \in \mathbb{R}^{r \times r}$, $V \in \mathbb{R}^{n \times r}$.
- Ahorra memoria y computo.

Ejemplo

Para A de 1000×20, SVD reducida requiere $U_r(1000 \times 20)$, $\Sigma_r(20 \times 20)$, $V(20 \times 20)$.

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Propiedades de la SVD

- Los valores singulares son unicos (salvo orden).
- $\|A\|_2 = \sigma_1$ (norma espectral).
- $\|A\|_F = \sqrt{\sum_{i=1}^{r} \sigma_i^2}$ (norma de Frobenius).
- El rango de A es el numero de valores singulares no nulos.
- Las columnas de U asociadas a $\sigma_i > 0$ forman base ortonormal del espacio columna.
- Las columnas de V asociadas a $\sigma_i > 0$ forman base ortonormal del espacio fila.

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Teorema de Eckart-Young-Mirsky

Mejor aproximacion de rango k

La matriz $A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top = U_k \Sigma_k V_k^\top$ es la mejor aproximacion de rango k a A en norma espectral o de Frobenius.

- U_k, V_k: primeras k columnas de U, V.
- Σ_k: submatriz diagonal con los k mayores valores singulares.

Aplicacion: Compresion de imagenes

- Una imagen en grises es una matriz $A \in \mathbb{R}^{h \times w}$.
- SVD: $A = U\Sigma V^\top$.
- Truncamos a k valores singulares: $A_k = U_k \Sigma_k V_k^\top$.
- Se almacenan $h \cdot k + k + k \cdot w$ numeros en lugar de $h \cdot w$.
- Mayor $k \Rightarrow$ mejor calidad, menor compresion.

svd_compresion.png

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Definicion mediante SVD

Pseudoinversa A^+

Si $A = U\Sigma V^\top$, entonces

$$A^+ = V\Sigma^+ U^\top,$$

donde Σ^+ se obtiene tomando el reciproco de cada $\sigma_i > 0$ y dejando ceros en los demas.

- $x = A^+ b$ minimiza $\|Ax - b\|_2$ y, entre las soluciones optimas, tiene la menor norma.
- Util para sistemas sobredeterminados o indeterminados.

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Compresion de imagenes (Vision por Computador)

- Se aplica SVD a cada canal de la imagen.
- Se descartan valores singulares pequeños (ruido).
- La imagen reconstruida A_k mantiene la estructura principal.
- Relacion compresion/calidad controlable via k.

Sistemas de recomendacion (Filtrado colaborativo)

- Matriz R de usuarios $\times$ productos (con valores faltantes).
- Se aproxima $R \approx UV^\top$ con $U \in \mathbb{R}^{m\times k}$, $V \in \mathbb{R}^{n\times k}$.
- La SVD truncada proporciona la mejor aproximacion de rango k en minimos cuadrados (rellenando ceros o usando metodos iterativos).
- Base del algoritmo ganador del Netflix Prize.

Analisis Semantico Latente (LSA) en NLP

- Matriz terminos-documentos A (tf-idf).
- SVD: $A = U\Sigma V^\top$.
- Las columnas de U representan terminos en espacio de conceptos.
- Las filas de $V^\top$ (o columnas de V) representan documentos.
- Truncar a k dimensiones agrupa terminos y documentos relacionados semanticamente.

Deep Learning

- **Inicializacion ortogonal**: usar U o V para inicializar pesos, preservando normas y gradientes.
- **Compresion de modelos**: aproximar capas fully connected grandes mediante SVD de bajo rango, reduciendo parametros y acelerando inferencia.
- **Pseudoinversa** en algunas arquitecturas (extreme learning machines).

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Calculo de SVD basico

```
import numpy as np

A = np.array([[1,2,3],[4,5,6],[7,8,9]], dtype=float)
U, s, Vt = np.linalg.svd(A, full_matrices=False)
print("U:\n", U)
print("Valores singulares:", s)
print("V^T:\n", Vt)

# Reconstruccion
Sigma = np.diag(s)
A_rec = U @ Sigma @ Vt
print("Error:", np.linalg.norm(A - A_rec))
```

Compresion de imagenes con SVD

```
import matplotlib.pyplot as plt
from skimage import data

img = data.camera().astype(float) / 255.0
U, s, Vt = np.linalg.svd(img, full_matrices=False)

k_values = [5,20,50,100]
plt.figure(figsize=(12,6))
for i,k in enumerate(k_values):
    Uk = U[:,:k]
    sk = s[:k]
    Vtk = Vt[:k,:]
    img_k = Uk @ np.diag(sk) @ Vtk
    plt.subplot(2,len(k_values),i+1)
    plt.imshow(img_k, cmap='gray')
    plt.title(f'k={k}'); plt.axis('off')
    plt.subplot(2,len(k_values),i+1+len(k_values))
    error = np.linalg.norm(img - img_k)/np.linalg.norm(img)
    plt.bar(i, error); plt.xticks([])
plt.tight_layout(); plt.show()
```

Pseudoinversa y regresion lineal

```
def pseudoinverse(A, tol=1e-10):
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    s_inv = np.array([1/v if v>tol else 0 for v in s])
    return Vt.T @ np.diag(s_inv) @ U.T

np.random.seed(42)
m,n = 20,5
A = np.random.randn(m,n)
x_true = np.random.randn(n)
b = A @ x_true + 0.1*np.random.randn(m)

x_pinv = pseudoinverse(A) @ b
x_lstsq, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
print("Error pseudoinv:", np.linalg.norm(x_pinv - x_true))
print("Error lstsq:", np.linalg.norm(x_lstsq - x_true))
```

Simulacion de sistema de recomendacion

```
n_users, n_items, k_true = 100, 80, 5
np.random.seed(42)
U_true = np.random.randn(n_users, k_true)
V_true = np.random.randn(n_items, k_true)
R = U_true @ V_true.T + 0.5*np.random.randn(n_users, n_items)
mask = np.random.rand(n_users, n_items) > 0.3
R_obs = R * mask

U, s, Vt = np.linalg.svd(R_obs, full_matrices=False)
k = 5
R_pred = U[:,:k] @ np.diag(s[:k]) @ Vt[:k,:]
test_mask = ~mask
rmse = np.sqrt(np.mean((R[test_mask] - R_pred[test_mask])**2))
print("RMSE en test:", rmse)
```

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Notacion comun

- $A = U\Sigma V^\top$: descomposicion en valores singulares.
- $A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$: aproximacion de rango k.
- σ_i: valores singulares.
- $A^+ = V\Sigma^+ U^\top$: pseudoinversa.
- En LSA: $X \approx T \cdot S \cdot D^\top$ (terminos, conceptos, documentos).

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Ejercicios

1. Calcula la SVD de la matriz $\begin{pmatrix} 0 & 2 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$ a mano.
2. Demuestra que $A^{+}A = V_r V_r^{\top}$ (proyeccion sobre espacio fila).
3. Usa SVD para comprimir una imagen a color (tres canales). Visualiza resultados con distintos k.
4. Implementa un sistema de recomendacion simple con SVD sobre MovieLens (subconjunto). Divide en train/test y evalua RMSE.
5. Investiga el algoritmo de "randomized SVD" y cuando es preferible usarlo.

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Problema

- Matriz terminos-documentos con valores singulares: $[200, 150, 80, 50, 30, 2, 1, 0{,}5, \ldots]$.
- Determinar rango efectivo.
- Visualizar documentos en 2D.
- Calcular varianza explicada por los primeros 2 y 5 componentes.

Solucion

a) Rango efectivo

Los primeros 5 valores son grandes; los siguientes son muy pequeños. Rango efectivo $= 5$ (se puede tomar un umbral, ej. ¿ 10).

b) Visualizacion

Se usan las primeras 2 columnas de V (vectores singulares derechos). Cada documento se representa por su fila en V_k.

c) Varianza explicada

Suma de cuadrados de los primeros k valores singulares / suma total. Total aproximado: $200^2 + 150^2 + 80^2 + 50^2 + 30^2 + \ldots \approx 73000$. Con 5: $40000 + 22500 + 6400 + 2500 + 900 = 72300$, $\approx 99\,\%$. Con 2: $40000 + 22500 = 62500$, $\approx 85{,}6\,\%$.

Contenido

1. Teorema de la Descomposicion en Valores Singulares
2. Propiedades Importantes
3. Aproximacion de Bajo Rango y Compresion
4. Pseudoinversa de Moore-Penrose
5. Aplicaciones en Inteligencia Artificial
6. Python: SVD con NumPy y Aplicaciones
7. Notacion en Papers Cientificos
8. Ejercicios Propuestos
9. Caso Practico: Analisis de Documentos con LSA
10. Resumen

Lo que hemos aprendido

Conceptos clave

- **SVD**: factorizacion $A = U\Sigma V^\top$, interpretacion geometrica.
- **Aproximacion de bajo rango**: $A_k = U_k \Sigma_k V_k^\top$ (Eckart-Young).
- **Pseudoinversa**: $A^+ = V\Sigma^+ U^\top$, solucion de minimos cuadrados.
- **Aplicaciones**: compresion de imagenes, sistemas de recomendacion, LSA, deep learning.
- **Python**: ‘np.linalg.svd‘, ‘lstsq‘, compresion, pseudoinversa.

Conexion con futuros cursos

- **Machine Learning**: PCA, LSA, recomendacion.
- **Vision por Computador**: compresion, analisis de componentes.
- **NLP**: espacios semanticos, recuperacion de informacion.
- **Deep Learning**: inicializacion ortogonal, compresion de modelos.

Gracias!